

Chapter 10

Virtual Memory

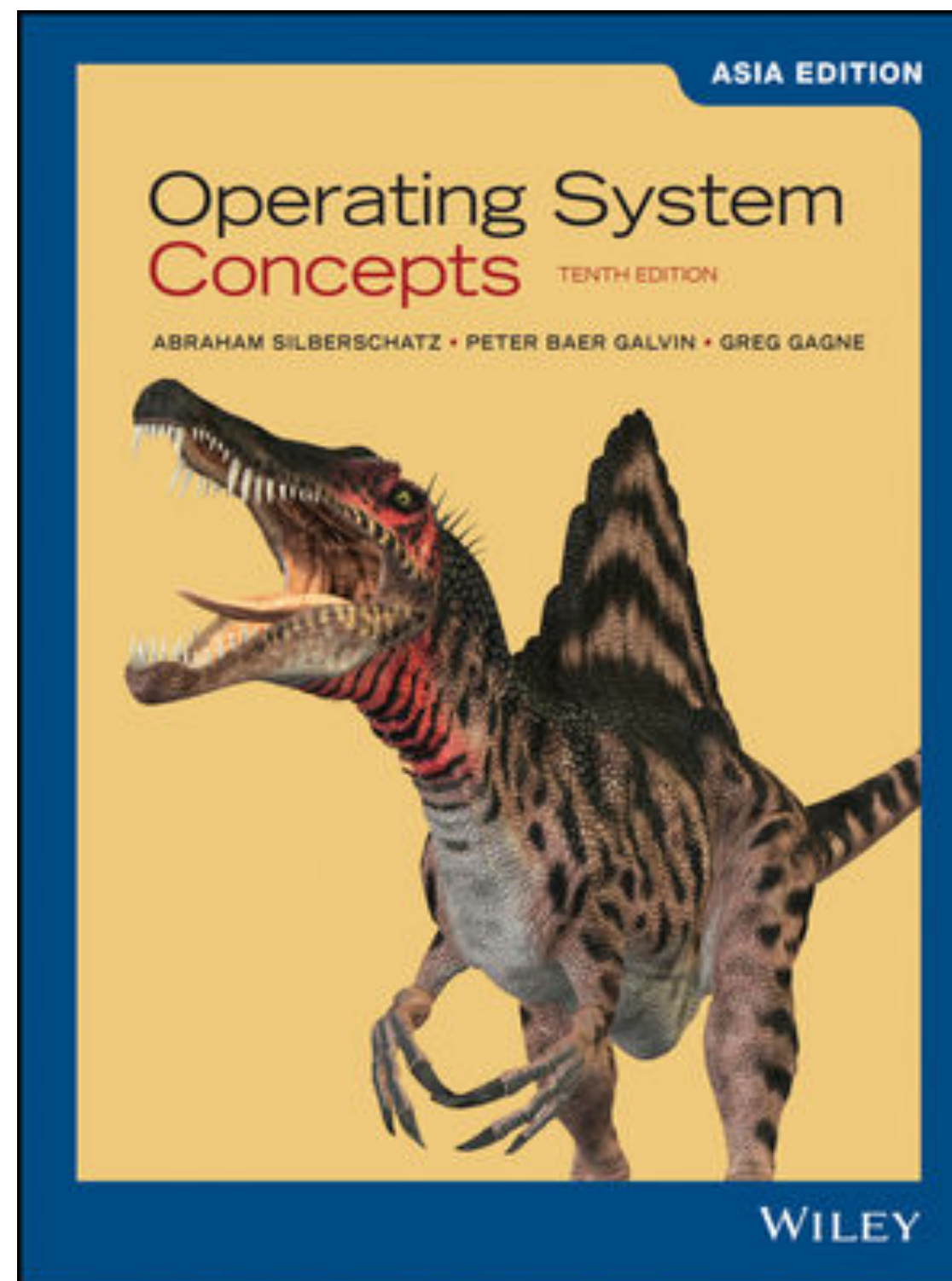
Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/7/29

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Outline

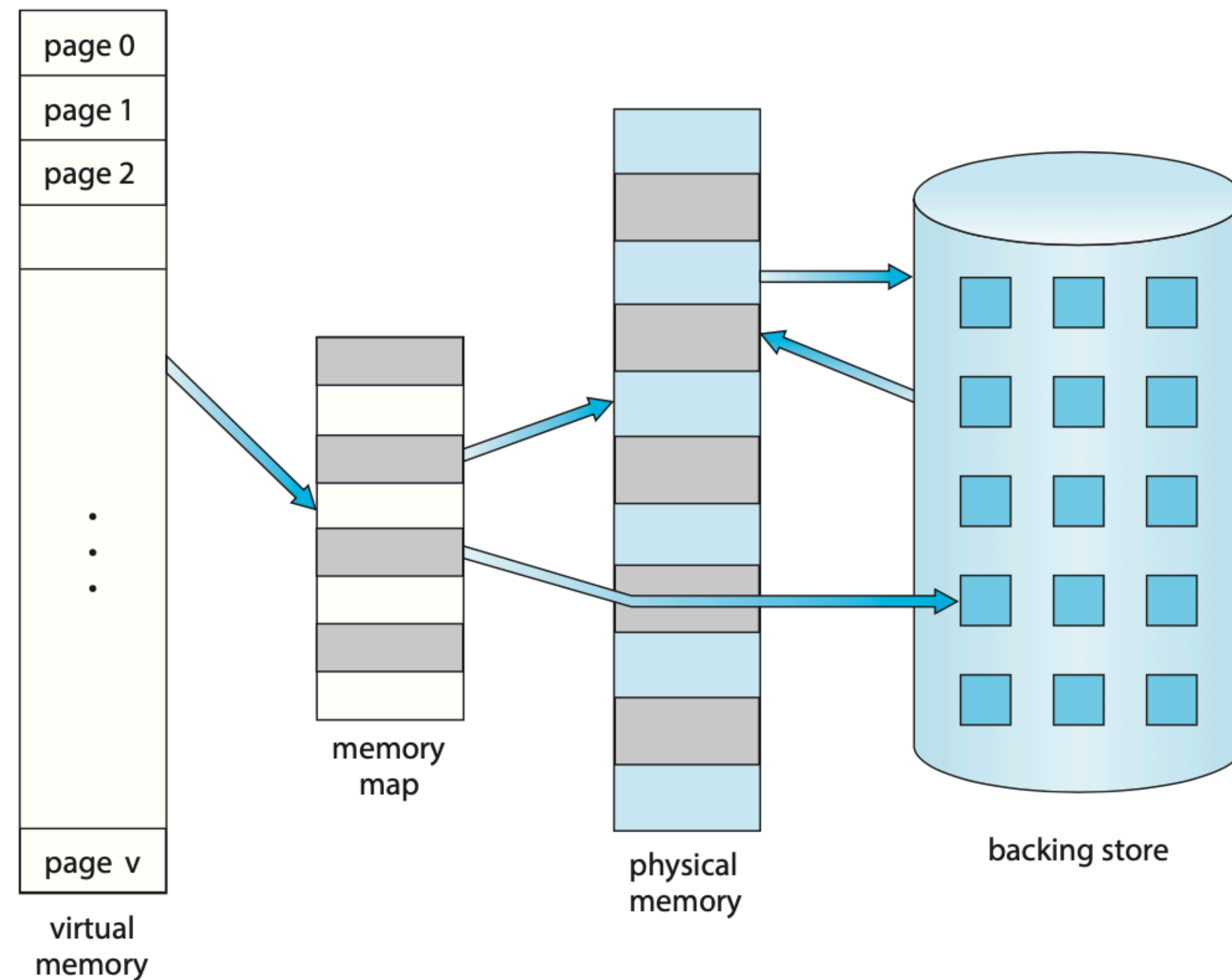
- Background
- Demand Paging
- Page Replacement
- Allocation of Frames
- Copy-on-Write

Outline

- Background
- Demand Paging
- Page Replacement
- Allocation of Frames
- Copy-on-Write

Background

- **Virtual memory** is a technique that allows the execution of processes that are **not completely in memory**.
- Programs can be larger than physical memory.



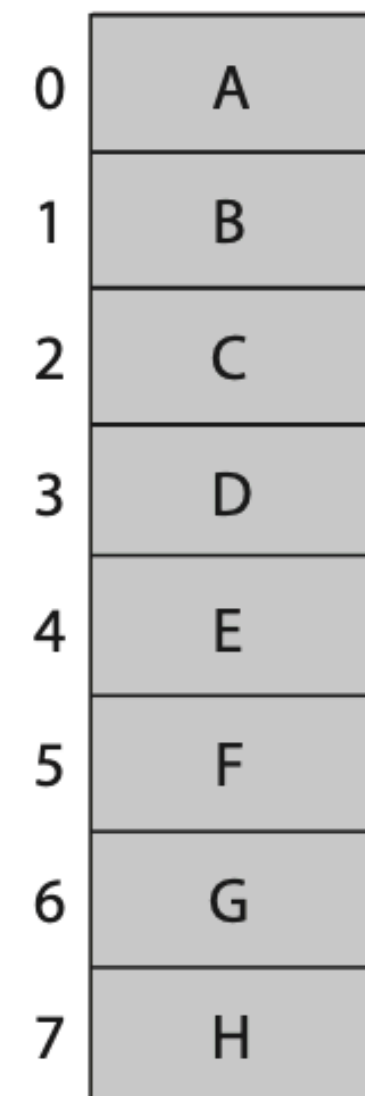
Outline

- Background
- Demand Paging
- Page Replacement
- Allocation of Frames
- Copy-on-Write

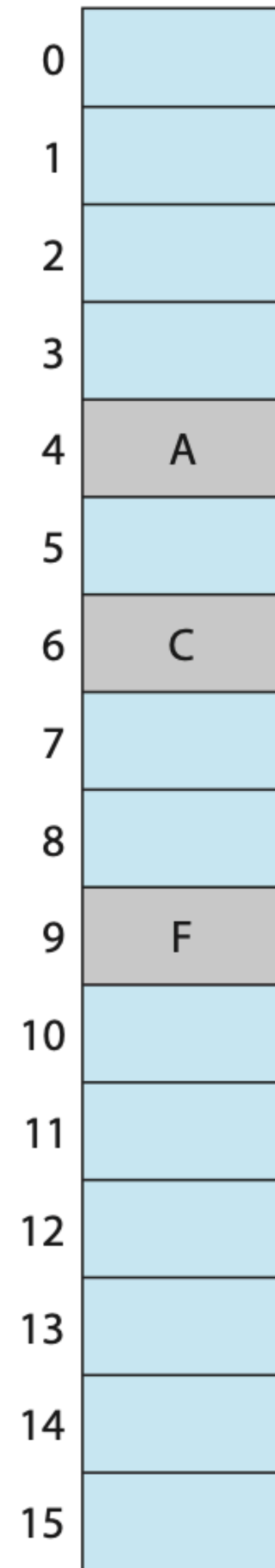
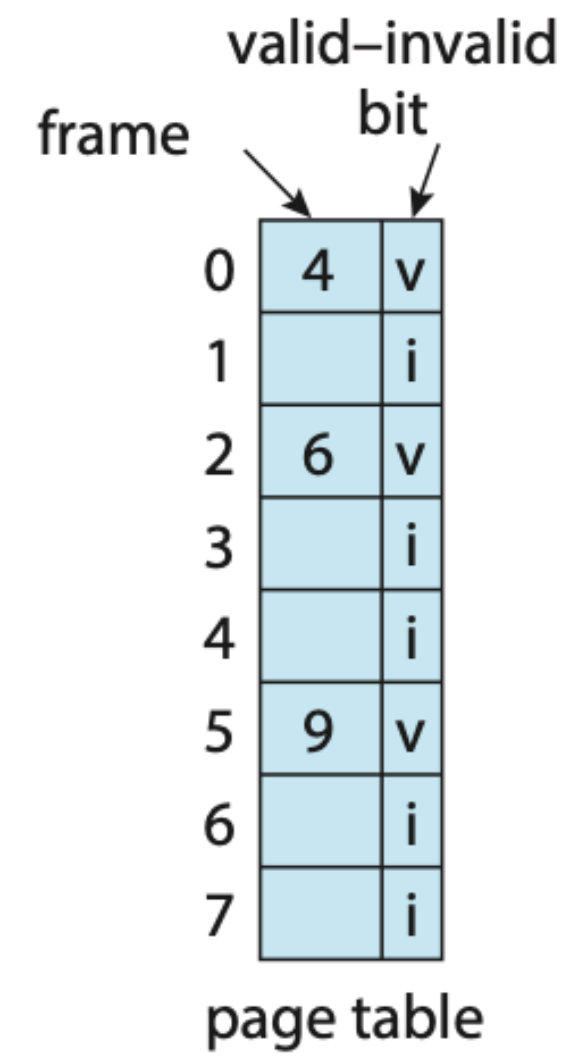
Demand Paging

- Load pages **only as they are needed**.
- With demand-paged virtual memory, pages are loaded only when they are **demanded** during program execution.
- A demand-paging system is similar to **a paging system with swapping**.

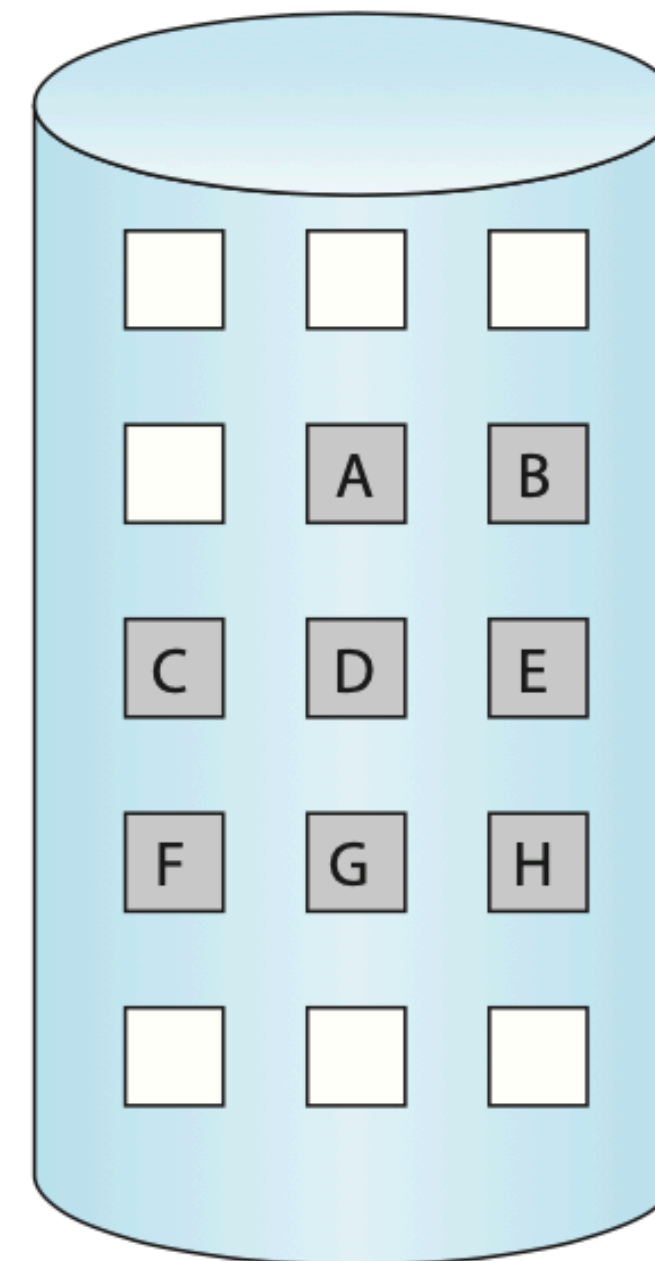
Basic Concepts



logical
memory



physical memory



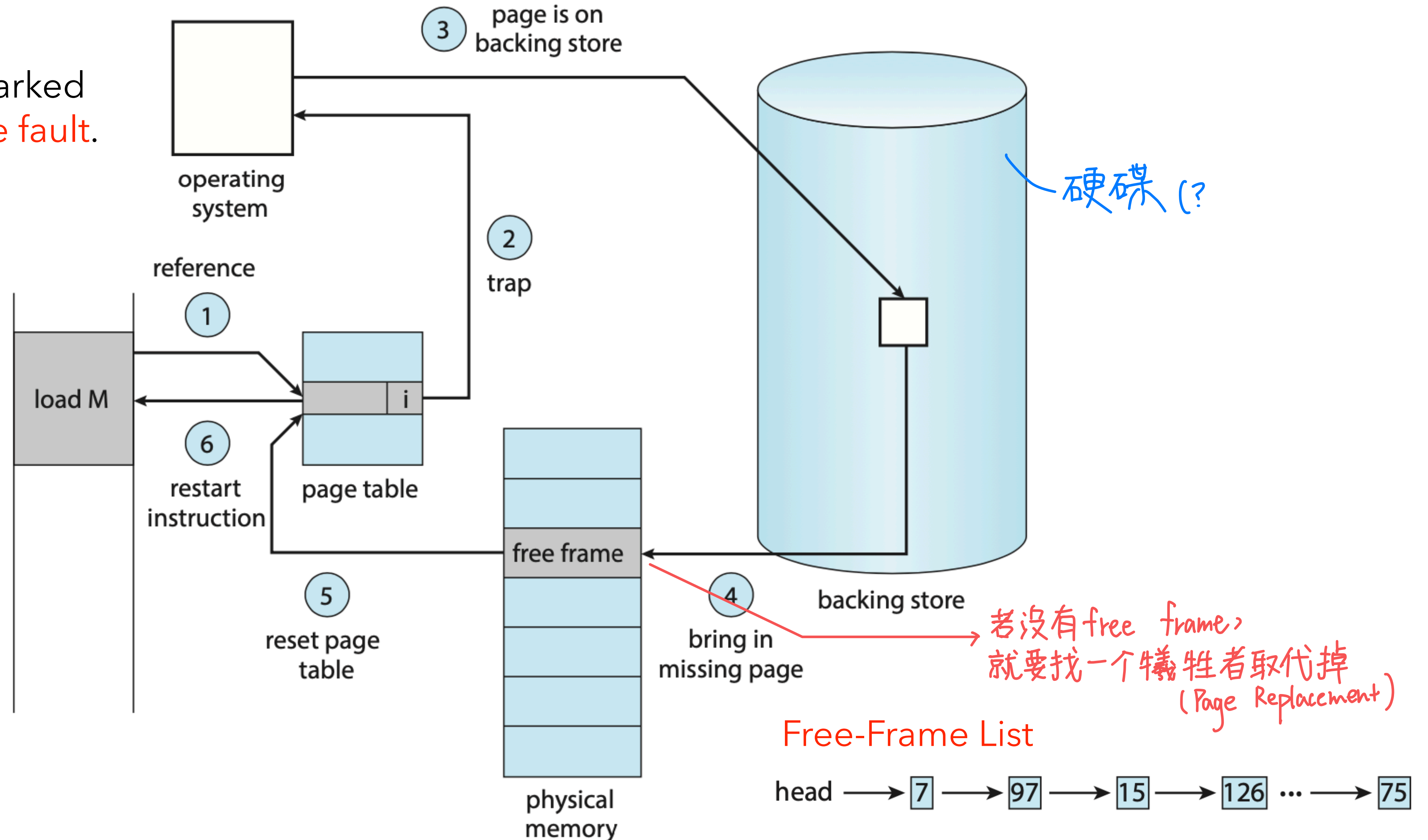
backing store

把硬碟當mem用)
把一些Page 放到硬碟中

Page Fault

Access to a page marked invalid causes a **page fault**.

當程式嘗試訪問的一個
記憶體空間尚不存在，還
在虛擬記憶體(硬碟)時
就會發生 Page Fault



Performance of Demand Paging

effective access time = $(1-p) * \text{memory-access time} + p * \text{page fault time}$

With an average page-fault time of 8 milliseconds and a memory-access time of 200 nanoseconds

$$\begin{aligned}\text{effective access time} &= (1-p)*200 + p*8000000 \\ &= 200 + 7999800*p\end{aligned}$$

$$\begin{aligned}200 + 7999800*p &< 220 \\ p &< 0.0000025\end{aligned}$$

Programs tend to have **locality of reference**.

Outline

- Background
- Demand Paging
- Page Replacement (記憶體用滿了, 要找一個犧牲者踢掉)
- Allocation of Frames
- Copy-on-Write

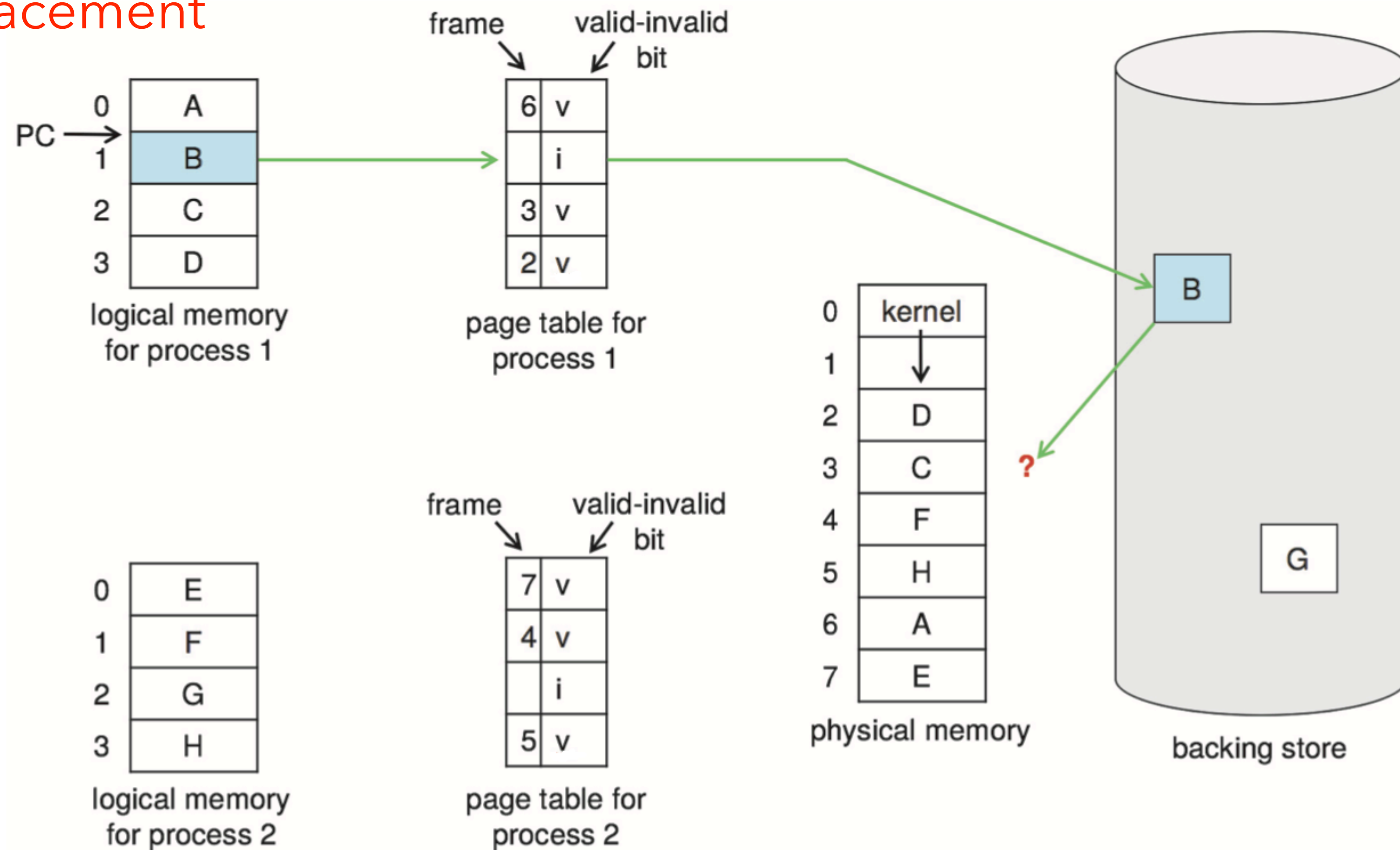
Page Replacement

- There are **no free frames** on the free-frame list. (**over-allocating**)

發生 Page Replacement
沒有 free frame!

- Terminate the process - not the best choice

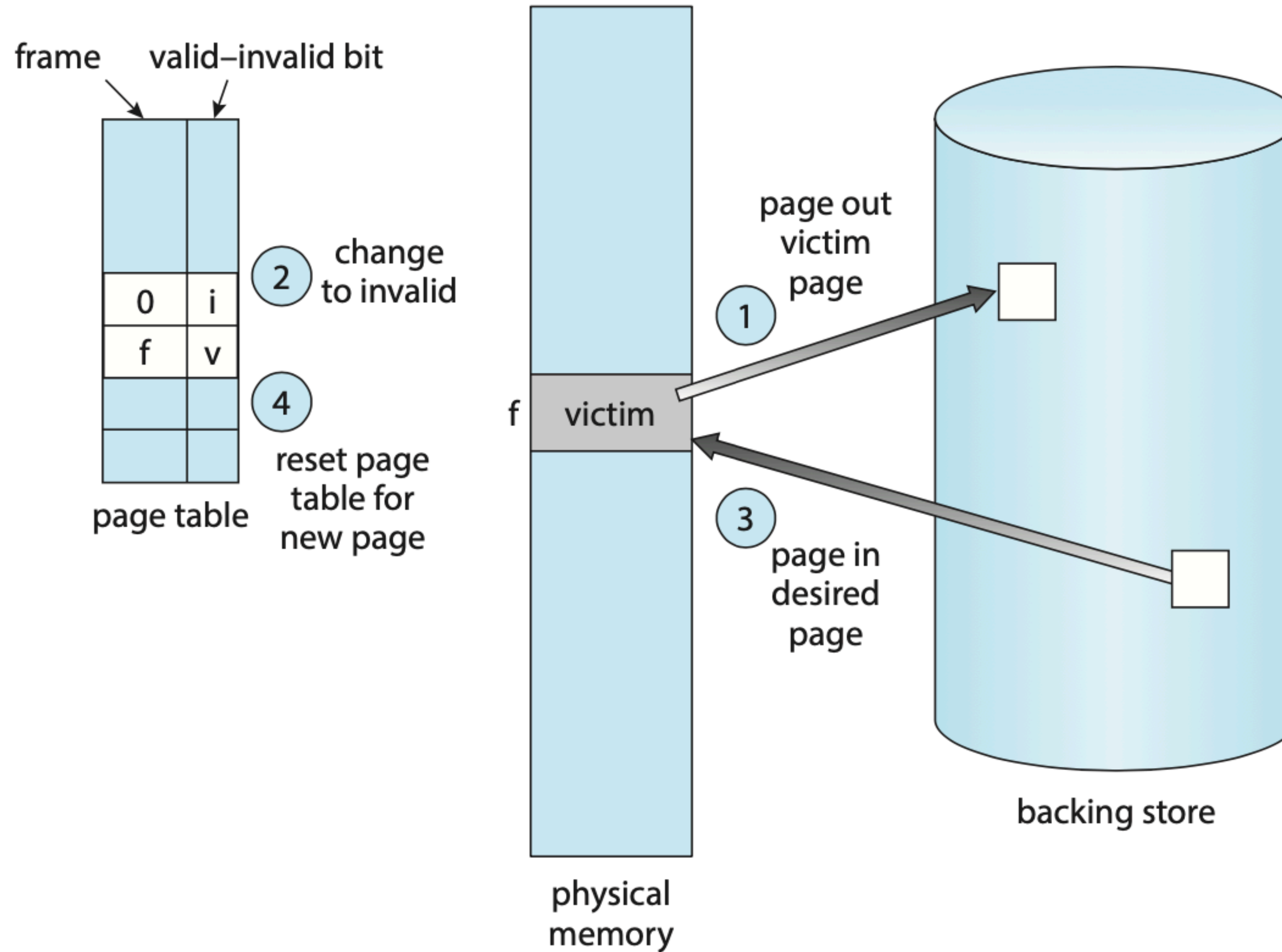
- Page replacement



Basic Page Replacement

- The page-fault service routine
 - Find the location of the **desired page** on secondary storage
 - Find a free frame:
 - If there is a free frame, use it.
 - If there is **no free frame**, use a **page-replacement algorithm** to **select a victim frame**
↘ 沒有 free-frame, 用演算法決定誰成為犧牲者
 - Write the victim frame to secondary storage (**if necessary**)(**modify/dirty bit**); change the page and frame tables accordingly
 - Read the desired page into the newly freed frame; change the page and frame tables
 - Continue the process from where the page fault occurred

Basic Page Replacement



Implementation of Demand Paging

- A page-replacement algorithm
 - Select the frames that are to be replaced
 - Section 10.4 Page Replacement
- A frame-allocation algorithm
 - How many frames to allocate to each process
 - Section 10.5 Allocation of Frames

Evaluation

- We evaluate a page-replacement algorithm by running it on a particular string of memory references and computing the number of page faults
- 使用 Page 的顺序
- Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- A memory with 3 frames (只有3个 frame 可用)

Algorithms

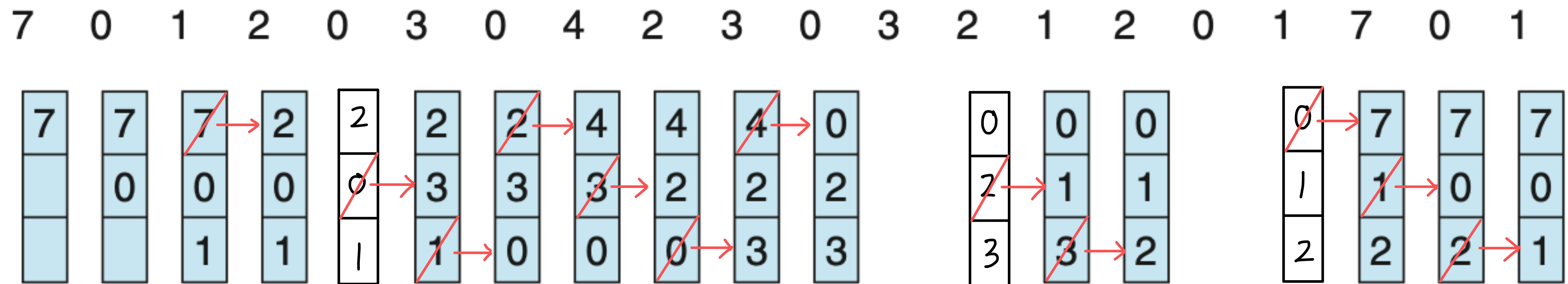
- FIFO Page Replacement
- Optimal Page Replacement
- LRU Page Replacement
- Counting-based Page Replacement
- Others

Algorithms

- FIFO Page Replacement
- Optimal Page Replacement
- LRU Page Replacement
- Counting-based Page Replacement
- Others

FIFO Page Replacement

- We create a FIFO (first-in, first-out) queue to hold all pages in memory
- Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- A memory with 3 frames



这边都有发生 Page Fault
(本来记忆体没有, 要 load 进来)

总共发生 15 次 Page Fault

FIFO Page Replacement

```
void FIFO(int reference[NUMBER_OF_REFERENCES], char memory[FRAMES]) {  
  
    int addedTime[FRAMES] = {0, 0, 0};  
    int faults = 0;  
  
    for (int time = 1; time <= NUMBER_OF_REFERENCES; time++) {  
  
        cout << reference[time-1] << ": ";  
        if (found(memory, reference[time-1]) == 1) {  
            cout << "\n";  
        } else {  
            int index = getOldestIndex(addedTime);  
            memory[index] = '0' + reference[time-1];  
            addedTime[index] = time;  
            show(memory);  
            faults++;  
        }  
    }  
    cout << "Number of Page Faults: " << faults << "\n";  
}
```

FIFO Page Replacement

```
int getOldestIndex(int time[FRAMES]) {
    int old = NUMBER_OF_REFERENCES + 1;
    int oldIndex = -1;
    for (int i = 0; i < FRAMES; i++) {
        if (time[i] < old) {
            old = time[i];
            oldIndex = i;
        }
    }
    return oldIndex;
}

int found(char memory[FRAMES], int page) {
    char target = '0' + page;
    for (int i = 0; i < FRAMES; i++) {
        if (target == memory[i]) {
            return 1;
        }
    }
    return 0;
}
```

FIFO Page Replacement

```
int main(int argc, const char * argv[]) {  
  
    int reference[NUMBER_OF_REFERENCES] = {7, 0, 1, 2, 0, 3, 0, 4, 2,  
        3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};  
  
    char memory[FRAMES] = {' ', ' ', ' '};  
  
    FIFO(reference, memory);  
  
    return 0;  
}
```

```
7: 7, ,  
0: 7, 0,  
1: 7, 0, 1  
2: 2, 0, 1  
0:  
3: 2, 3, 1  
0: 2, 3, 0  
4: 4, 3, 0  
2: 4, 2, 0  
3: 4, 2, 3  
0: 0, 2, 3  
3:  
2:  
1: 0, 1, 3  
2: 0, 1, 2  
0:  
1:  
7: 7, 1, 2  
0: 7, 0, 2  
1: 7, 0, 1
```

Number of Page Faults: 15

FIFO Page Replacement (Belady's anomaly)

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Belady's anomaly: for some page-replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.

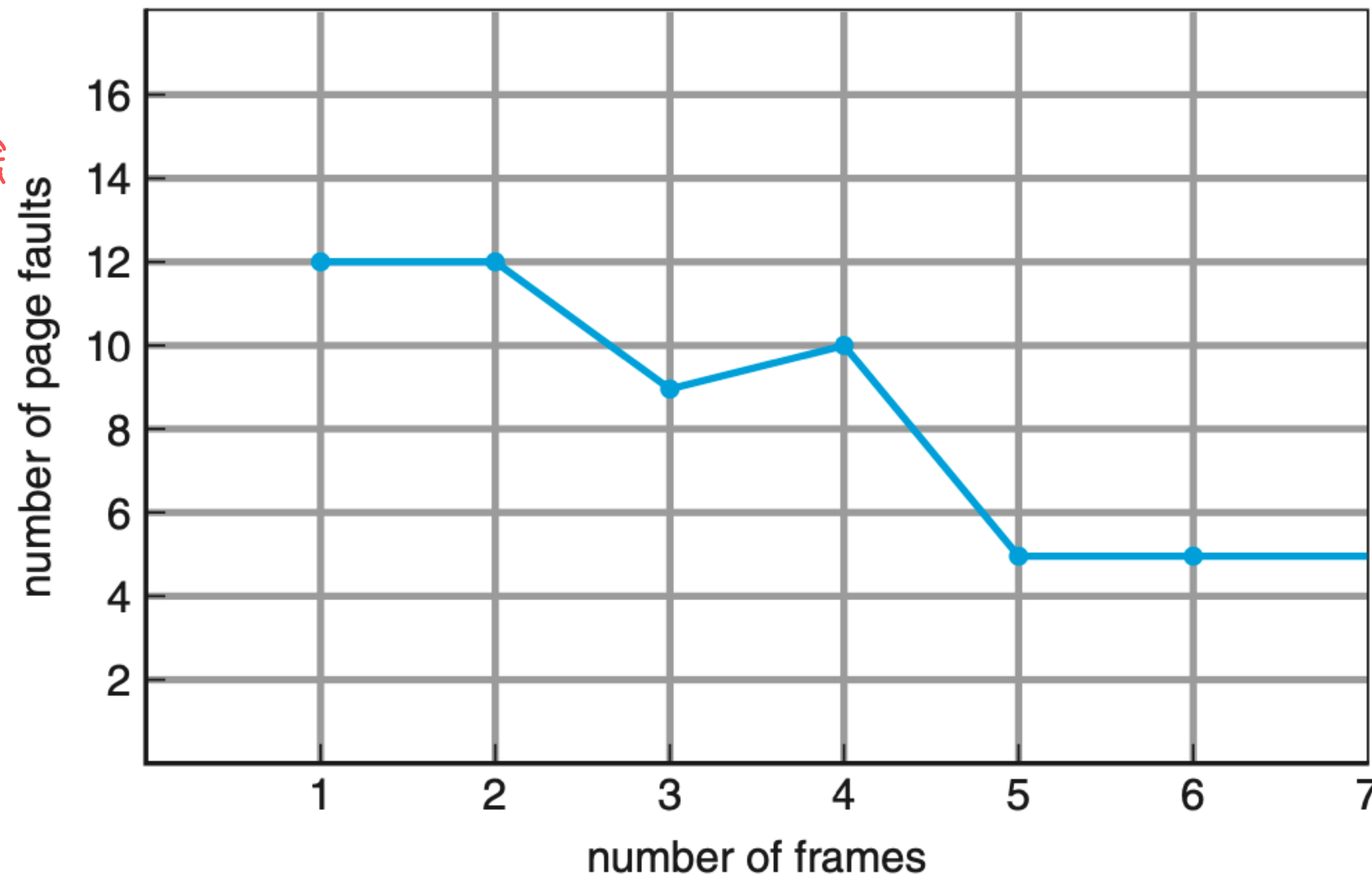
Belady's anomaly:

給越多的Frame，
發生Page Fault的機會
反而會增加。

↓ e.g. FIFO Algo
在此 Reference Str 時

3 Frame: 9

4 Frame: 10



Algorithms

- FIFO Page Replacement
- Optimal Page Replacement
- LRU Page Replacement
- Counting-based Page Replacement
- Others

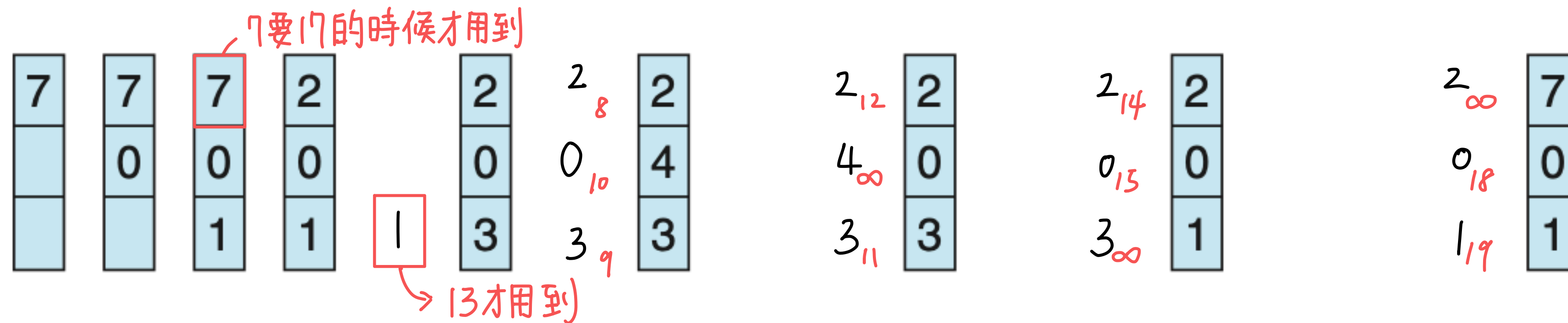
Optimal Page Replacement

最佳的 Page Replacement Algo (理想, 無法實現) (看未來)

- Replace the page that **will not** be used for the **longest** period of time. (把最晚才會用到的 frame 取代掉)
- This algorithm (**OPT** / **MIN**) has the lowest page-fault rate and will never suffer from Belady's anomaly.
- Unfortunately, it requires **future knowledge** of the reference string

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



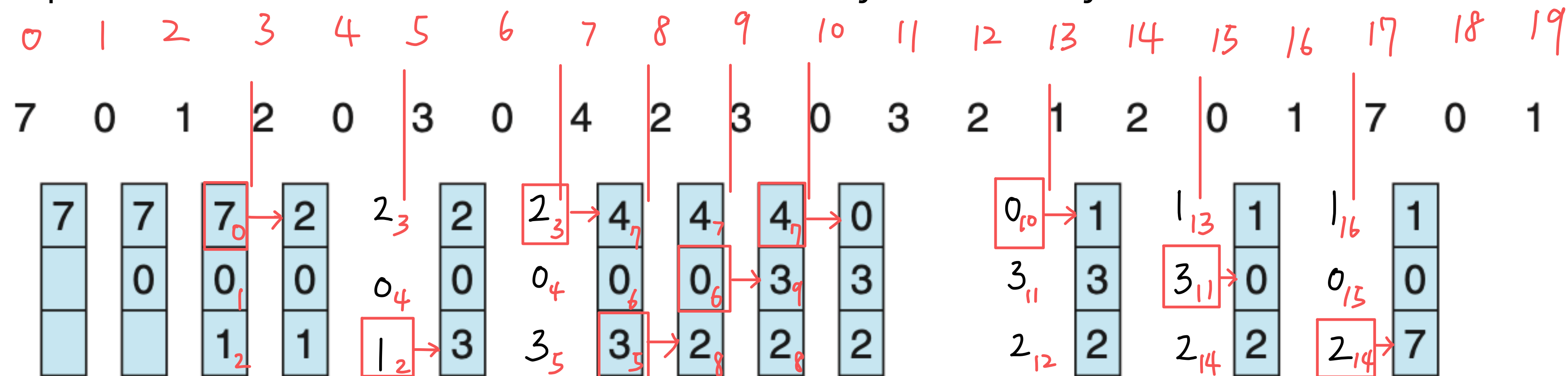
Algorithms

- FIFO Page Replacement
- Optimal Page Replacement
- LRU Page Replacement 最近最少被用到
- Counting-based Page Replacement
- Others

LRU Page Replacement

挑最近最少被用到的(看过去)

- Replace the page that **has not been used** for the longest period of time.
- Least Recently Used (LRU)
- The LRU policy is often used as a page-replacement algorithm and is considered to be good.
- LRU replacement does **not suffer from Belady's anomaly**



Optimal : 9次 Page Fault
LRU : 12次 Page Fault

Linux

✦ AI 摘要

Linux employs a sophisticated page replacement mechanism that is primarily based on the Least Recently Used (LRU) principle, but with significant modifications and enhancements to optimize performance in real-world scenarios.

Instead of a strict LRU implementation, the Linux kernel utilizes active and inactive LRU lists.

- The `active_list` contains pages that are considered part of the current working set of processes and are frequently accessed.
- The `inactive_list` holds pages that are candidates for reclamation (swapping out or freeing) if memory pressure arises. Pages in the inactive list are less recently used than those in the active list.

When a page needs to be replaced (e.g., due to a page fault and no free memory), the kernel prioritizes reclaiming pages from the inactive list. If a page in the inactive list is accessed again, it is moved back to the active list, effectively giving it a "second chance" and preventing premature eviction.

Algorithms

- FIFO Page Replacement
- Optimal Page Replacement
- LRU Page Replacement ↑ 只考到這裡
- Counting-based Page Replacement
- Others

CH6~CH10 (CH7不出題、不考)

期末會有 spin lock 相關的
程式題

Tronclass 上 PageReplacement.CPP 會考
└ LRU, FIFO, optimal

ContiguousMemory.CPP 會考

└ first-fit, worst fit
best-fit (Chapter 9)

Critical Section, Race Condition 解釋

內、外部斷裂 CH9

Peterson solution, Semaphore, spinlock, Deadlock

Counting-based Page Replacement

- We can keep a counter of the number of references that have been made to each page.
 - The least frequently used (LFU) (最不常用的換掉)
 - The most frequently used (MFU) (最常用的換掉)
 - The page with the smallest count was probably just brought in and has yet to be used
- > 透過計數 (counting) 的方式實現!

Algorithms

- FIFO Page Replacement
- Optimal Page Replacement
- LRU Page Replacement
- Counting-based Page Replacement
- Others

Others

- LRU-Approximation Page Replacement
 - Additional-Reference-Bits Algorithm
 - Second-Chance Algorithm

給兩次機會，第二次才換掉

Outline

- Background
- Demand Paging
- Page Replacement
- Allocation of Frames
- Copy-on-Write

Implementation of Demand Paging

- A page-replacement algorithm
 - Select the frames that are to be replaced
 - Section 10.4 Page Replacement
- A frame-allocation algorithm
 - How many frames to allocate to each process
 - Section 10.5 Allocation of Frames

Minimum Number of Frames

- There are various constraints to the strategies for the allocation of frames
 - You cannot allocate more than the total number of available frames
 - At least a minimum number of frames should be allocated to each process.
每个 Process 都有一个最低 frame 的保障
 - The first reason is, as less number of frames are allocated, there is an increase in the page fault ratio, decreasing the performance of the execution of the process
 - Secondly, there should be enough frames to hold all the different pages that any single instruction can reference

Allocation Algorithms

- Equal allocation
- Proportional allocation

Equal Allocation

- Split m frames among n process
- Give each process m/n frames
- Problem: Allocation of a large number of frames to a **small process** will eventually lead to the **wastage** of a large number of allocated unused frames.

Proportional Allocation

- We allocate available memory to each process according to its size
- Let the size of the virtual memory for process p_i be s_i , and $S = \sum s_i$
- Then, if the total number of available frames is m , we allocate a_i frames to p_i , where $a_i = (s_i/S)*m$

Global versus Local Allocation (Replacement)

- **Global Replacement** allows a process to select a replacement frame from the set of all frames, even if that frame is currently allocated to some **other** processes
 - Advantage: Does not hinder the performance of processes
 - Disadvantage: The page fault ratio of a process can not be solely controlled by the process itself. The pages in memory for a process depends on the paging behavior of other processes as well.

Global versus Local Allocation (Replacement)

- **Local Replacement:** the number of frames allocated to a process does not change
 - Advantage: The pages in memory for a particular process and the page fault ratio is affected by the paging behavior of only that process.
 - Disadvantage: A low priority process may hinder a high priority process by not making its frames available to the high priority process.

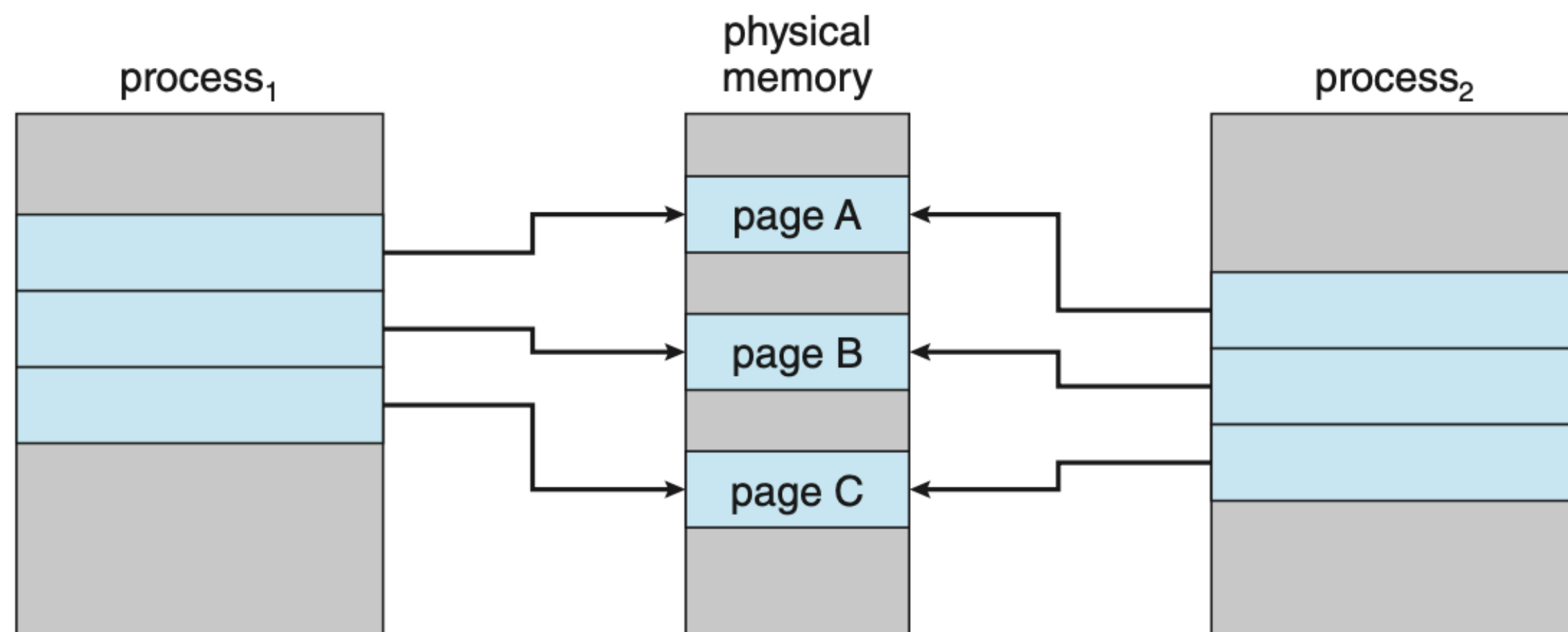
Outline

- Background
- Demand Paging
- Page Replacement
- Allocation of Frames
- Copy-on-Write

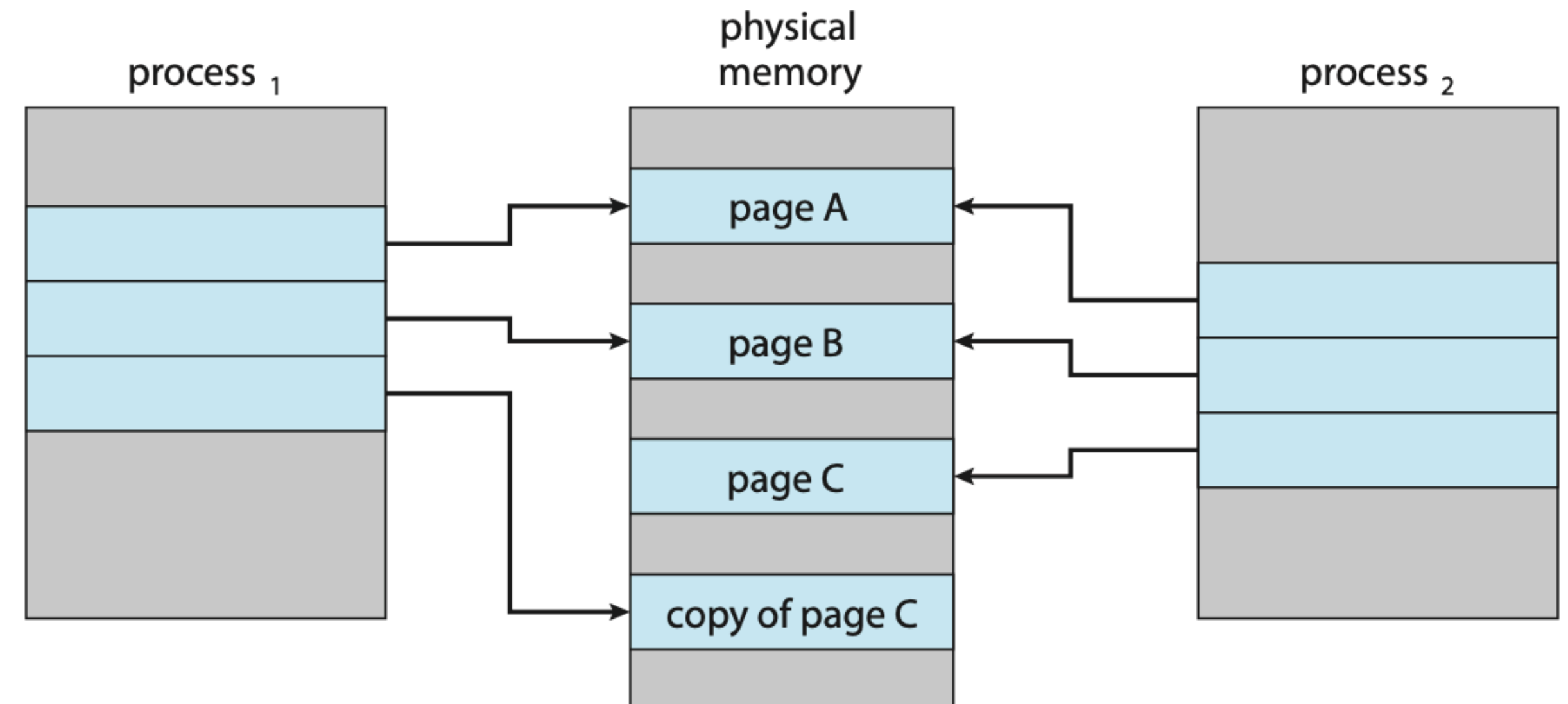
Copy-on-Write

- Copy-on-Write (COW) is only used in systems that support virtual memory
- Map multiple virtual pages to the same physical memory page.
- Allowing the parent and child processes initially to share the same pages. These shared pages are marked as copy-on-write pages

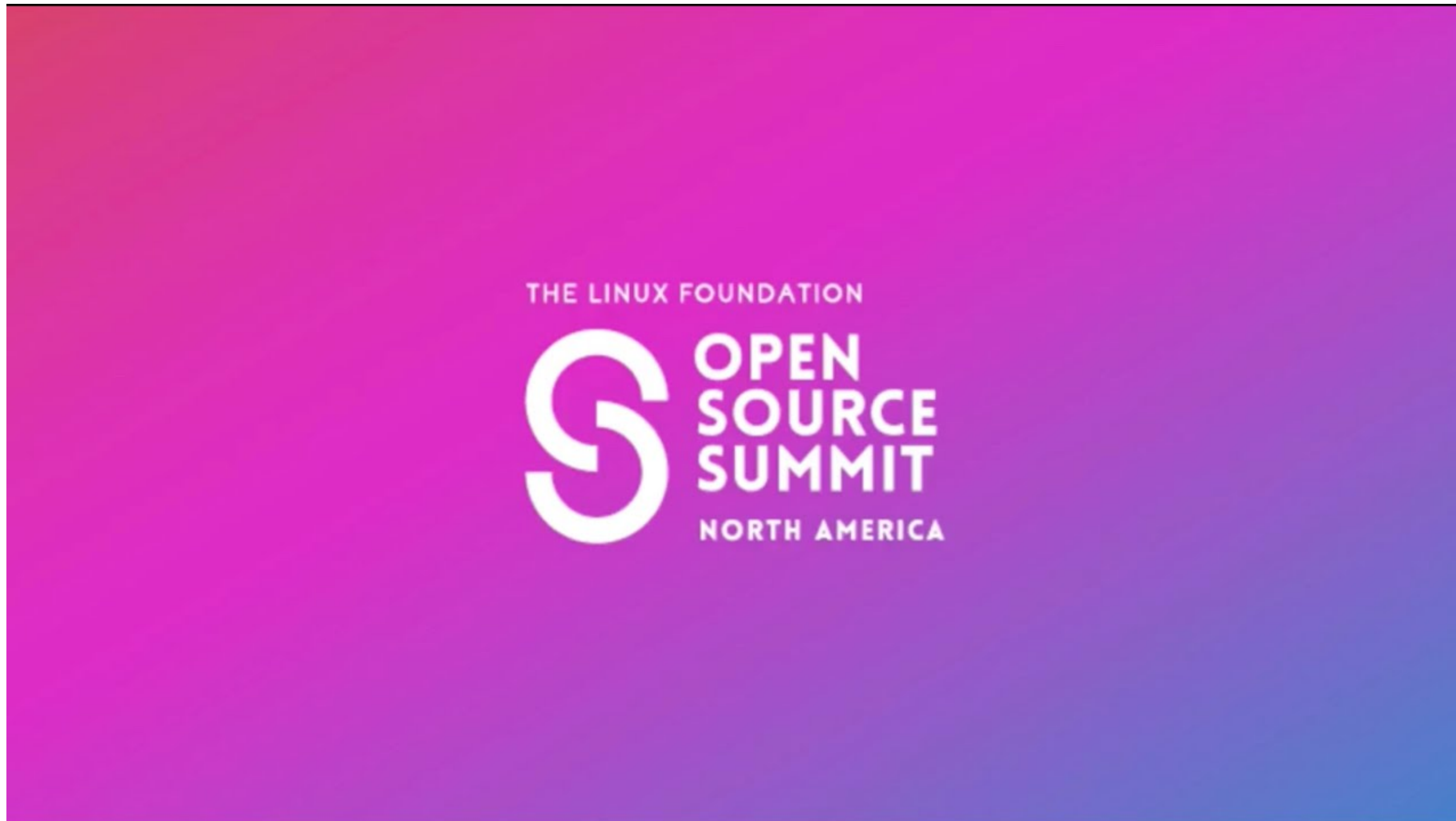
Before process 1 modifies page C



After process 1 modifies page C



NTOU Student



<https://www.youtube.com/watch?v=J53PxYfpro4>